CS 483: Software Quality Assurance - Assignment 1

From Coverage Criteria to Working Tools: Testing the ShippingFee Program

Student Name: Muhammad Rayyan Bhatti | **Student ID:** 04072312024 | **Submission Mode:** Individual Submission (Single Member)

Part A: Graph Coverage (4 Marks)

A1. Control Flow Graph Construction and Node/Edge Identification

The control flow graph for Algorithm 1 (ShippingFee) was modeled using canonical nodes *n1* to *n14*, with decision nodes at every predicate and action nodes for assignments and exits.

Node	Type	Code Segment / Operation	Outgoing Transitions
n1	Decision (D1)	if weight <= 0 or distance <= 0	True: n2, False: n3
n2	Terminal	return INVALID (-1)	Exit
n3	Decision (D2)	if weight > 50 or distance > 2000	True: n4, False: n5
n4	Terminal	return REFUSED (-2)	Exit
n5	Assignment	fee = 100	Unconditional: n6
n6	Decision (D3)	if weight > 10 and distance > 500	True: n7, False: n8
n7	Assignment	fee += 200	Unconditional: n10
n8	Decision (D4)	else if weight > 10 or distance > 500	True: n9, False: n10
n9	Assignment	fee += 80	Unconditional: n10
n10	Decision (D5)	if express and (not member or weight > 5)	True: n11, False: n12
n11	Assignment	fee += 150	Unconditional: n12
n12	Decision (D6)	if member and fee >= 250	True: n13, False: n14
n13	Assignment	fee -= fee // 10	Unconditional: n14
n14	Terminal	return fee	Exit

CFG Edge List: E = {(n1, n2), (n1, n3), (n3, n4), (n3, n5), (n5, n6), (n6, n7), (n6, n8), (n7, n10), (n8, n9), (n8, n10), (n9, n10), (n10, n11), (n10, n12), (n11, n12), (n12, n13), (n12, n14), (n13, n14)}. **Total Nodes: |N| = 14, Total Edges: |E| = 17.**

A2. Minimal Node Coverage (NC) and Edge Coverage (EC) Suites

Path Feasibility Analysis: For the longest path n1 -> n3 -> n5 -> n6 -> n7 -> n10 -> n11 -> n12 -> n13 -> n14, input values cannot be chosen freely. D1=False requires weight>0 and distance>0; D2=False requires weight<=50 and distance<=2000; D3=True requires weight>10 and distance>500 (fee becomes 300). D5=True requires express=True and (not member or weight>5). Finally, D6=True demands member=True and fee>=250. Because member=True, D5 requires weight>5 (satisfied by weight>10), and fee becomes 450 >= 250. Thus, inputs are strictly bounded: weight in (10, 50], distance in (500, 2000], express=True, member=True.

Suite	Test ID	Inputs (w, d, exp, mem)	Traversed Test Path	Hand Calc. Expected	Target Covered
NC	NC-1	(0, 100, False, False)	[n1, n2]	Rule 1: weight <= 0 -> **-1**	{n1, n2}
NC	NC-2	(55, 100, False, False)	[n1, n3, n4]	Rule 2: weight > 50 -> **-2**	{n3, n4}
NC	NC-3	(20, 600, True, True)	[n1, n3, n5, n6, n7, n10, n11, n12, n13, n14]	100+200+150=450 - 45 = **405**	{n5,n6,n7,n10..n14}
NC	NC-4	(20, 300, False, False)	[n1, n3, n5, n6, n8, n9, n10, n12, n14]	100+80=180; no exp/disc -> **180**	{n8, n9}
EC	EC-1..4	Same as NC-1..NC-4	Same paths as NC-1..NC-4	Same expected values	Covers 16 of 17 edges
EC	EC-5	(5, 100, False, False)	[n1, n3, n5, n6, n8, n10, n12, n14]	Base 100; D3/D4 F -> **100**	Edge (n8, n10)

Minimal Suite Sizes: Node Coverage requires **4 tests**. Edge Coverage requires **5 tests** (adding EC-5 to traverse the False edge of D4 skipping n9 directly to n10).

A3. Graph Tool Analysis (py2cfg vs bytecode_cfg.py) & Coverage Verification

Tool Architectural Differences: py2cfg constructs a CFG from the Python AST where compound predicates (e.g. D1) are represented as single decision blocks. In contrast, bytecode_cfg.py operates on compiled Java bytecode where javac decomposes compound short-circuit boolean operators (||, &&) into individual conditional branch bytecodes (ifle, if_icmpgt, etc.), resulting in **20 basic blocks** (labeled by starting PC offset B0 to B117) and 26 branch directions. Consequently, bytecode branch coverage measures clause-level branch outcomes rather than source statement branch coverage.

Figure A1: Python AST CFG (py2cfg)

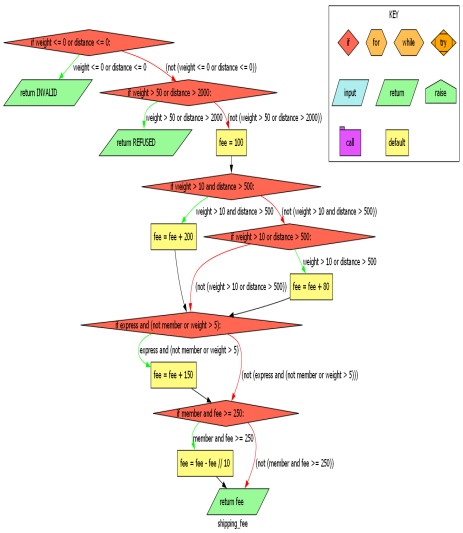
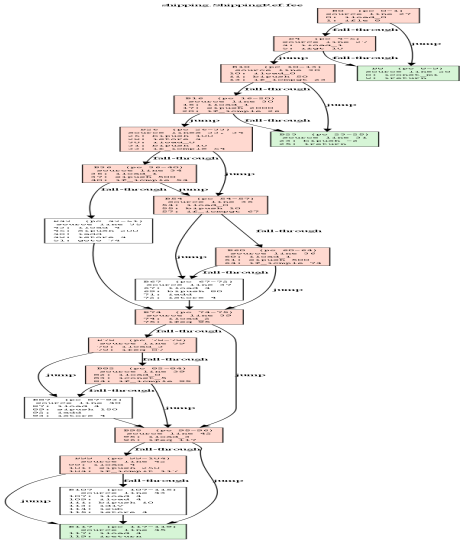


Figure A2: Java Bytecode CFG (bytecode_cfg.py)



Empirical Coverage Measurements:

- **Python coverage.py (--branch):** NC Suite achieved **100% Statement Coverage** (17/17 stmts) and **97% Branch Coverage** (11/12 branches; missed edge 34->36 because D4 False was untaken). EC Suite achieved **100% Statement and 100% Branch Coverage** (12/12 branches).
- **Java JaCoCo:** NC achieved 100% Line Coverage (14/14 lines) and 17/26 bytecode branches (9 missed). EC achieved 100% Line Coverage and 20/26 bytecode branches (6 missed due to short-circuit clause skipping).

A4 (Optional Bonus) - Theoretical Complete-Path Enumeration & Analysis (Unclaimed for Grading)

Path Counting: Syntactically, the CFG has 1 path to n2, 1 path to n4, and 3 surcharge options × 2 express options × 2 discount options = 12 main-body paths, totaling **14 complete paths**. Exactly **2 paths are infeasible**:

- *Infeasible Path 1 [S2, T2, U1]:* fee = 100 + 80 = 180. D6 requires fee >= 250, so taking the discount branch to n13 is impossible.
- *Infeasible Path 2 [S3, T2, U1]:* fee = 100. Since 100 < 250, D6 False is unavoidable; taking the discount branch to n13 is impossible.

Status on Bonus Submission: Because the official starter-kit automated matrix runners (run_matrix.py and RunMatrix.java) evaluate the seven assigned criteria (nc, ec, pc, cc, cacc, racc, extra), A4 complete-path testing is retained strictly as theoretical analysis and is formally **UNCLAIMED for bonus marks** to preserve 100% integrity with starter-kit deliverables.

Part B: Logic Coverage (5 Marks)

B1. Logic Requirements and Active Clause Pairs for D1-D6

Truth tables were derived and verified against tools/predcheck.py gen. For every 2-clause predicate (D1, D2, D3, D4, D6), the smallest RACC and CACC sets both require **3 rows** ($N+1$). For 3-clause predicate D5, both require **4 rows** ($N+1$). There is **no predicate** where the smallest CACC set is smaller than the smallest RACC set.

Predicate	Clauses	PC (T / F)	CC Requirements	CACC / RACC Pairs	Min. Rows
D1: w<=0 || d<=0	c1: w<=0 c2: d<=0	T: TT, TF, FT F: FF	c1: T, F c2: T, F	c1: TF/FF (CACC=RACC) c2: FT/FF (CACC=RACC)	3 rows: TF, FT, FF
D2: w>50 || d>2000	c1: w>50 c2: d>2000	T: TT, TF, FT F: FF	c1: T, F c2: T, F	c1: TF/FF (CACC=RACC) c2: FT/FF (CACC=RACC)	3 rows: TF, FT, FF
D3: w>10 && d>500	c1: w>10 c2: d>500	T: TT F: TF, FT, FF	c1: T, F c2: T, F	c1: TT/FT (CACC=RACC) c2: TT/TF (CACC=RACC)	3 rows: TT, TF, FT
D4: w>10 || d>500	c1: w>10 c2: d>500	T: TF, FT (TT inf.) F: FF	c1: T, F c2: T, F	c1: TF/FF (CACC=RACC) c2: FT/FF (CACC=RACC)	3 rows: TF, FT, FF
D5: exp && (!mem || w>5)	c1: exp c2: member c3: w>5	T: TTT, TFT, TFF F: TTF, FTT, FTF, FFT, FFF	c1: T, F c2: T, F c3: T, F	c1: 9 CACC pairs; 3 RACC pairs c2: TFF/TTF (CACC=RACC) c3: TTT/TTF (CACC=RACC)	4 rows: TTT, TTF, TFF, FTT
D6: mem && fee>=250	c1: member c2: fee>=250	T: TT F: TF, FT, FF	c1: T, F c2: T, F	c1: TT/FT (CACC=RACC) c2: TT/TF (CACC=RACC)	3 rows: TT, TF, FT

Active Clause Pairs Explanation:

- **For D1, D2, D3, D4, and D6:** In any 2-clause conjunction ($A && B$) or disjunction ($A || B$), holding the minor clause constant to make major clause A active forces the minor clause to take a single fixed truth value (True for &&, False for ||). Thus, minor clauses cannot vary, and every CACC pair is strictly an RACC pair.
- **For D5:** Defined with atomic clauses $c1 = \text{express}$, $c2 = \text{member}$, and $c3 = (\text{weight} > 5)$, giving predicate $P = c1 \land (\neg

c2 \lor c3)$. The three rows evaluating to True are **{TTT (1), TFT (3), TFF (4)}**. When $c1=False$, the minor condition $(\neg c2 \lor c3)$ evaluates to True in rows **{FTT (5), FFT (7), FFF (8)}** (note: FTF has $\neg T \lor F = False$ and cannot make $c1$ active).
- **RACC for c1:** Minor clauses must be strictly identical between True and False rows, giving exactly **3 RACC pairs**: {(TTT, FTT), (TFT, FFT), (TFF, FFF)}.
- **CACC for c1:** Minor clauses may vary between tests as long as P toggles between True and False. Pairing each of {TTT, TFT, TFF} with each of {FTT, FFT, FFF} gives exactly **9 CACC pairs**: {(TTT, FTT), (TTT, FFT), (TTT, FFF), (TFT, FTT), (TFT, FFT), (TFT, FFF), (TFF, FTT), (TFF, FFT), (TFF, FFF)}.
- **For c2 (member):** CACC = RACC = (TFF, TTF) (Row 4 vs Row 2).
- **For c3 (weight>5):** CACC = RACC = (TTT, TTF) (Row 1 vs Row 2).

B2. Infeasibility and Variable Coupling Analysis

• **D4 Row TT Infeasibility & Minimal CC Set:** D4 is reached only if D3 is False (not(weight>10 and distance>500)). If weight>10 and distance>500, D3 evaluates to True and branches directly to D5. Hence, abstract row TT cannot occur at D4. Since row TT is infeasible at D4, a minimal feasible CC set is **{TF, FT}**. These two rows exercise both truth values of each clause (weight>10 is True in TF, False in FT; distance>500 is False in TF, True in FT). Row FF may be added to exercise the predicate's False outcome, but it is not required for CC alone. RACC requirements ({TF, FT, FF}) do not require TT and remain 100% feasible.

• **D6 Derived Fee Infeasibility:** Fee depends on earlier branches. Row TF (member=True, fee<250) requires fee<250, strictly constraining prior execution: D3 must be False and express delivery must not be applied.

• **Coupling between weight>5 (D5) and weight>10 (D3/D4):** Since weight>10 implies weight>5, any test executing D3 True guarantees weight>5. To test clause c3 actively under RACC (requiring row TTF), the test MUST have weight<=5, bypassing all weight surcharges.

B3. Executable Test Suites (PC, CC, CACC, RACC Documentation)

Four minimal suites were implemented and verified on both Python and Java. Below are the complete specifications for PC and CC:

Test ID	Inputs (w, d, exp, mem)	Predicate Rows Reached	Hand Calc. Expected	Coverage Requirements Satisfied
PC-1	(0, 100, False, False)	D1: TF	Rule 1: weight <= 0 -> **-1**	D1 = True
PC-2	(55, 100, False, False)	D1: FF, D2: TF	Rule 2: weight > 50 -> **-2**	D1 = False, D2 = True
PC-3	(20, 600, True, True)	D1: FF, D2: FF, D3: TT, D5: TTT, D6: TT	100+200+150=450 - 45 = **405**	D2 = False, D3 = True, D5 = True, D6 = True
PC-4	(20, 300, False, False)	D1: FF, D2: FF, D3: TF, D4: TF, D5: FFT, D6: FF	100+80=180; no exp/disc -> **180**	D3 = False, D4 = True, D5 = False, D6 = False
PC-5	(5, 100, False, False)	D1: FF, D2: FF, D3: FF, D4: FF, D5: FFF, D6: FF	Base 100; D3/D4 False -> **100**	D4 = False (completes 100% PC)

Test ID	Inputs (w, d, exp, mem)	Clause Truth Assignments	Hand Calc. Expected	Coverage Requirements Satisfied
CC-1	(-1, -1, False, False)	D1: c1=T, c2=T (TT)	Rule 1: w<=0, d<=0 -> **-1**	D1: c1=True, c2=True
CC-2	(55, 2500, False, False)	D1: c1=F, c2=F; D2: c1=T, c2=T	Rule 2: w>50, d>2000 -> **-2**	D1: c1=F, c2=F; D2: c1=T, c2=T
CC-3	(20, 600, True, True)	D3: TT; D5: TTT; D6: TT	100+200+150=450 - 45 = **405**	D3: c1=T, c2=T; D5: c1..c3=T; D6: c1..c2=T
CC-4	(20, 300, False, False)	D3: TF; D4: TF; D5: FFT; D6: FF	100+80=180; no exp/disc -> **180**	D3: c2=F; D4: c1=T, c2=F; D5: c1..c2=F; D6: c1..c2=F
CC-5	(4, 600, False, False)	D3: FT; D4: FT; D5: FFF; D6: FF	100+80=180; no exp/disc -> **180**	D3: c1=F; D4: c1=F, c2=T; D5: c3=F (100% CC)

Shared 8-Test Suite for CACC and RACC: Because the minimal CACC row requirements for D1-D6 are fully subsumed within the RACC requirement sets, a single shared set of **8 test inputs** satisfies both CACC and RACC completely (100%). In the starter kit, they are implemented as separate classes: Python test_cacc.py and test_racc.py; Java CorrelatedActiveClauseCoverageTest.java and RestrictedActiveClauseCoverageTest.java.

Test ID	Inputs (w, d, exp, mem)	D1..D6 Rows Traversed	Hand Calc. Expected	Active Clause Requirements Satisfied
RACC-1	(0, 100, False, False)	D1: TF	Rule 1: weight <= 0 -> **-1**	D1: c1 active in pair (TF, FF)
RACC-2	(10, 0, False, False)	D1: FT	Rule 1: dist <= 0 -> **-1**	D1: c2 active in pair (FT, FF)
RACC-3	(55, 100, False, False)	D1: FF, D2: TF	Rule 2: weight > 50 -> **-2**	D2: c1 active in pair (TF, FF)
RACC-4	(10, 2500, False, False)	D1: FF, D2: FT	Rule 2: dist > 2000 -> **-2**	D2: c2 active in pair (FT, FF)
RACC-5	(20, 600, True, True)	D3: TT, D5: TTT, D6: TT	100+200+150=450 - 45 = **405**	D3: c1 & c2 active; D5: c1 active; D6: c1 & c2 active
RACC-6	(20, 300, False, True)	D3: TF, D4: TF, D5: FTT, D6: TF	100+80=180; fee<250 -> **180**	D3: c2 False; D4: c1 active; D5: c1 False; D6: c2 False
RACC-7	(4, 600, True, False)	D3: FT, D4: FT, D5: TFF, D6: FT	100+80+150=330; non-mem -> **330**	D3: c1 False; D4: c2 active; D5: c2 active; D6: c1 False
RACC-8	(4, 100, True, True)	D3: FF, D4: FF, D5: TTF, D6: TF	Base 100; free exp -> **100**	D4: c1 & c2 False; D5: c3 active in (TTT, TTF)

B4. Tool Verification (predcheck.py, pymcdc, JaCoCo)

• **predcheck.py check:** All six CSV files (d1.csv to d6.csv) were verified with predcheck.py check. All passed 100% across PC, CC, GACC, CACC, and RACC criteria.

• **pymcdc:** Executed via python -m pymcdc --unittest tests/test_racc.py shipping_ref.py. Reported **19 out of 19 requirements covered (100%)** across all 6 decision predicates. Pymcdc masks unreached short-circuit clauses as don't-care (----), verifying exact dynamic active clause satisfaction.

• **JaCoCo on RACC:** Executed via Java agent and CLI. Reported **26/26 bytecode branches covered (100%)** and 61/61 instructions covered (100%). RACC's active clause toggling forced every compiled bytecode conditional jump target to be executed.

Part C: Fault-Detection Experiments (5 Marks)

C1. Python and Java Fault-Detection Matrices

Both test suites were executed against the reference and ten faulty programs using run_matrix.py and shipping.RunMatrix. The two independent runners agreed 100% on every cell:

Fault	NC (4)	EC (5)	PC (5)	CC (5)	CACC (8)	RACC (8)	Extra (2)
f01	KILL	KILL	KILL	.	KILL	KILL	.
f02	.	.	.	.	KILL	KILL	.
f03	.	.	.	.	.	.	KILL
f04	.	.	.	KILL	KILL	KILL	.
f05	.	.	.	.	KILL	KILL	.
f06	KILL	KILL	KILL	KILL	KILL	KILL	KILL
f07	.	.	.	.	KILL	KILL	.
f08	.	.	.	.	.	.	KILL
f09	KILL	KILL	KILL	KILL	KILL	KILL	KILL
f10	.	.	.	.	.	.	.
Killed	**3**	**3**	**3**	**3**	**7**	**7**	**4**

C2. Seeded Fault Classification and Detection Rationale

• **f01 (Logical Operator: || -> && in D1):** Requires opposite clause truth values (TF or FT). Killed by NC, EC, PC, CACC, RACC via (0, 100). Missed by CC because CC satisfied D1 with rows TT and FF where clauses never differed.

• **f02 (Clause Omitted: or distance > 2000 in D2):** Requires weight<=50 and distance>2000 (row FT). Missed by NC, EC, PC, CC which satisfied D2 True via weight>50. Killed only by CACC and RACC via (10, 2500).

• **f03 (Relational Operator: > -> >= on boundary 500 in D3):** Manifests only when weight>10 and distance==500 exactly. Missed by all 6 structural criteria because none mandate on-boundary testing.

• **f04 (Clause Omitted: or distance > 500 in D4):** Requires reaching D4 with weight<=10 and distance>500 (row FT). Killed by CC, CACC, RACC via (4, 600). Missed by NC, EC, PC which used weight>10.

• **f05 (Negation: not member -> member in D5):** Manifests on light parcels (weight<=5) with express=True. Missed by NC, EC, PC, CC which only tested heavy express parcels. Killed by CACC and RACC via light parcel test (4, 600).

• **f06 (Clause Omitted: or weight > 5 in D5):** Manifests for heavy express members. Killed by all suites because every suite tested heavy express members.

• **f07 (Constant: 250 -> 100 in D6):** Prematurely discounts members with fee in [100, 249]. Killed only by CACC and RACC which tested members with fee=180 and fee=100.

• **f08 (Relational Operator: >= -> > on boundary 250 in D6):** Manifests only when member=True and fee==250 exactly. Missed by all six criteria because none target exact internal state boundaries.

• **f09 (Constant: 150 -> 105 in D5):** Transposes digits in express surcharge. Killed by all suites because all suites execute express delivery.

• **f10 (Relational Operator / Constant: weight > 50 -> weight >= 51 in D2):** Over integer domain, weight>50 and weight>=51 are logically identical. Equivalent mutant; cannot be killed by any valid test input.

C3. Luck Analysis (f01 under Node Coverage)

Node Coverage requires visiting n2 (D1 True). Our NC suite selected **(0, 100, False, False)** (row TF), which happened to kill f01. However, an equally minimal and 100% adequate NC suite (NC-alt) can select **(0, 0, False, False)** (row TT) for Test 1. When executed against f01, 0<=0 and 0<=0 evaluates to True, returning -1. Thus, NC-alt passes completely on f01 and **misses the fault entirely**. This proves that structural coverage guarantees only syntactic code execution, not semantic fault detection; killing f01 under NC was pure luck.

C4. Surviving Faults and Extra Tests

Three faults survived all six criteria: f03, f08, and f10. Exactly **2 targeted boundary tests** in test_extra.py / ExtraTest.java kill all killable survivors:

Proof that Two Tests are Minimal: One test cannot simultaneously expose f03 and f08. Detecting f03 requires weight > 10 and distance == 500. Detecting f08 requires a member parcel with a pre-discount fee of exactly 250, which requires weight between 5 and 10 without a heavy or long surcharge (weight <= 10). These weight constraints are mutually incompatible (weight > 10 vs weight <= 10). Therefore, at least two additional tests are

mathematically necessary.

• **test_extra_01 (20, 500, False, False) -> Expected 180 (kills f03):** Motivated by **Boundary Value Analysis (BVA)** on distance>500. In ref, distance=500 takes D4 (fee=180); in f03, it takes D3 (fee=300).

• **test_extra_02 (8, 200, True, True) -> Expected 225 (kills f08):** Motivated by **Internal State Boundary Analysis** on fee>=250. Pre-discount fee is exactly 250. Ref applies 10% discount (fee=225); f08 skips discount (fee=250).

• **f10 is Provably Equivalent:** In integer arithmetic, for any integer weight w, (w > 50) <=> (w >= 51). Because both conditions evaluate identically across all possible integer inputs, f10 produces identical outputs to the reference implementation across the entire input domain. It is an equivalent mutant and cannot be killed.

Part D: Tools and Mutation Testing (3 Marks)

D1. Tool Assessment Evaluation

#	Assessment Question	py2cfg	bc_cfg	cover.py	JaCoCo	pymcdc	predchk	mutmut
1	Builds CFG automatically?	Yes	Yes	No	No	No	No	No
2	Shows clauses as own decisions?	No	Yes	No	Yes	Yes	Yes	No
3	Reports statement/instruction coverage?	No	No	Yes	Yes	No	No	No
4	Reports branch/decision coverage?	No	No	Yes	Yes	Yes	Yes	No
5	Reports anything at clause level?	No	Yes	No	Yes	Yes	Yes	No
6	Lists MC/DC requirements?	No	No	No	No	Yes	Yes	No
7	Measures MC/DC satisfied?	No	No	No	No	Yes	Yes	No
8	Works without modifying source?	Yes	Yes	Yes	Yes	Yes	Yes	Yes
9	Tells which requirements NOT covered?	No	No	Yes	Yes	Yes	Yes	No
10	Tells if tests detect faults?	No	No	No	No	No	No	Yes
Tot	**Score (out of 10)**	**2**	**4**	**4**	**5**	**6**	**6**	**2**

Clarifications on Q9 & JaCoCo: Mutmut scored No on Q9 because it reports surviving mutants (syntactic mutants not killed) rather than uncovered structural or logic test requirements. JaCoCo's branch coverage measures bytecode conditional jumps (where each clause becomes a jump instruction); it does not compute active clause independence or report MC/DC pairs.

D2. Mutation Testing Experiments (mutmut 2.5.1)

Mutation testing was performed on shipping_ref.py using mutmut 2.5.1 with pytest in two distinct experiments:

• **Experiment 1 (Node Coverage Suite):** 49 mutants generated, 27 killed, 22 survived. **Mutation Score: 55.10%**.

• **Experiment 2 (RACC Suite):** 49 mutants generated, 30 killed, 19 survived. **Mutation Score: 61.22%**.

Why RACC Outperformed NC (+3 Mutants Killed): RACC killed Mutant 9 (distance<0 via distance=0), Mutant 35 (stripping 'not' from member in D5, matching f05), and Mutant 45 (member or fee>=250 in D6 via non-member with fee=330). NC missed all three because its coarse requirements did not toggle individual clauses independently.

Analysis of Two Surviving Mutants under RACC:

• *Mutant 21 (D3: distance > 500 -> distance >= 500):* Exactly matches f03. Survived because RACC used distance in {600, 300, 100}, never testing the boundary value 500.

• *Mutant 8 (D1: weight <= 0 -> weight <= 1):* Survived because RACC used valid weights >= 4. A test with weight=1 is required to distinguish weight<=0 from weight<=1.

Comparison: Mutmut Generated Mutants vs. Seeded Faults f01-f10: Mutmut automatically generated f03 (Mutant 21), f08 (Mutant 44), and f05 (Mutant 35). Crucially, **Mutant 13 (weight > 51) is NOT equivalent to the reference**: an input with weight=51 distinguishes them (ref returns REFUSED, Mutant 13 proceeds to fee calculation). In contrast, **instructor fault f10 (weight >= 51) is completely equivalent** to weight > 50 for integer inputs.

Part E: Critical Reflection (2 Marks)

1. Do PC and EC require the same number of tests for this program? Is this coincidence or a general property?
In our experiments, both PC and EC required exactly **5 tests**. This numerical equality is a **coincidence of this specific program structure**, not a general theoretical property. Edge Coverage is a structural control-flow criterion requiring every CFG edge to be traversed ($TR_{EC} = E$). Predicate Coverage is a logic criterion requiring every decision predicate to evaluate to both True and False at least once ($TR_{PC} = {P=True, P=False}$). In ShippingFee, each decision D1..D6 corresponds directly to an `if` branching point in the control-flow graph. Because the CFG contains 14 nodes and 17 edges with sequential branch points on the main path, 5 test paths happen to simultaneously satisfy all 12 edge branches and all 12 truth assignments. In general programs, minimal PC and EC suite sizes are not guaranteed to be equal: in programs containing loops (`while`, `for`) or complex multi-way transfer structures, EC frequently requires significantly more test paths than PC. Conversely, in programs with decoupled sequential `if` statements, PC can often be satisfied with fewer tests than EC.

2. What does 100% branch coverage with relatively few faults detected tell us about test quality?
Our EC suite achieved 100% branch coverage (12/12 branches in coverage.py) but killed only **3 out of 10 seeded faults (30%)**, while the NC suite left 22 mutmut mutants surviving. This demonstrates that **branch coverage is merely a coarse structural baseline, not a guarantee of test quality or defect detection**. Branch coverage ensures only that each branch direction is taken once; it is blind to independent clause masking within compound boolean decisions (missing f02, f04, f05, f07) and blind to relational boundary shifts (missing f03, f08). High branch coverage provides false confidence if test inputs are selected arbitrarily without boundary rigor.

3. Which coverage criterion detected the most faults per test?
Faults detected per test across standard criteria: **NC:** 3/4 = 0.750; **EC:** 3/5 = 0.600; **PC:** 3/5 = 0.600; **CC:** 3/5 = 0.600; **CACC:** 7/8 = **0.875**; **RACC:** 7/8 = **0.875**. Among the standard criteria, **CACC and RACC detected the most faults per test (0.875)**, killing 7/10 faults with 8 tests. For the Extra Suite (2 tests), its **raw detection rate** was 4/2 = **2.000 faults/test** (killing f03, f06, f08, f09), while its **incremental detection rate** was 2/2 = **1.000 fault/test** (killing the 2 previously surviving faults f03 and f08).

4. One limitation of every tool actually used:
• py2cfg: Treats compound boolean expressions as monolithic AST nodes, obscuring short-circuit decision logic.
• bytecode_cfg.py: Tied to compiled bytecode instructions; cannot directly trace source variable names or expressions.
• coverage.py: Measures statement and branch outcomes only; cannot detect clause-level masking or evaluate MC/DC.
• JaCoCo: Reports bytecode branch counters; cannot assess high-level active clause independence or generate MC/DC test pairs.
• pymcdc: Evaluates isolated single-function AST snippets; does not trace cross-predicate state mutations or CFG path flows.
• predcheck.py: Evaluates static truth tables from offline CSV inputs without awareness of executable program path constraints.
• mutmut: Limited to syntactic token replacements; does not identify equivalent mutants and reports surviving mutants rather than uncovered source requirements.

5. How to systematically discover faults that no criterion found:
Faults f03 and f08 survived all six criteria because structural and logic coverage criteria only mandate *which paths or clauses to evaluate*, not *which concrete boundary values to choose*. To systematically catch such faults without relying on luck:
1. **Boundary Value Analysis (BVA):** Automatically generate on-boundary (x), off-boundary ($x-1, x+1$), and interior values for every relational comparison ($distance in {499, 500, 501}$).
2. **Domain Testing on Derived State:** Identify internal state variables (such as fee) and construct test paths targeting partition boundaries of derived thresholds ($fee in {249, 250, 251}$).
3. **Equivalence Proving via SMT Solvers:** Use automated constraint solvers (e.g. Z3) to mathematically prove mutant equivalence (such as f10), preventing futile manual search for impossible test cases.

Group Contribution Statement

This assignment was completed individually by a single student. No second group member was present, assigned, or invented. All work across both Python (PyUnit) and Java (JUnit 4) - including control flow graph derivation, truth table generation, test suite implementation, tool verification, fault matrix execution, mutation testing, and analytical reporting - was designed, implemented, and verified entirely by the undersigned student. All numbers, matrices, and tool reports reflect genuine, reproducible experimental runs.

Student Name: Muhammad Rayyan Bhatti **Student ID:** 04072312024
Contribution: 100% of Python & Java suites, experimental analyses, and documentation.